# ACTIVITY 7: Creating Account Subclasses (Savings & Current)

## Objective

Create specialized account types by extending the base `Account` class with inheritance.

## Why Inheritance?

- **Code Reuse**: Common functionality in parent class
- **Specialization**: Each account type has unique rules
- **Polymorphism**: Treat different accounts uniformly
- **Extensibility**: Easy to add new account types later

## Requirements

### Architecture

text

```
Account (Parent)
    ├─── SavingsAccount (Child)
    └─── CurrentAccount (Child)
```

### Parent Class Changes

Make the `Account` class abstract with:

- Abstract method: `double getMinimumBalance()`
- Abstract method: `String getAccountType()`
- Constructor validates common rules (age, minimum balance via abstract method)
- All other methods remain the same

## SavingsAccount Class

- Minimum balance: ₹500
- Interest rate: 4% per annum
- Additional method: `calculateInterest(int years)`

## CurrentAccount Class

- Minimum balance: ₹1000
- Overdraft facility: ₹5000 limit
- Additional method: `getOverdraftLimit()`
- Additional method: `getAvailableOverdraft()`

## Key Changes from Base Account

| Aspect | Base Account | Savings Account | Current Account |
|---|---|---|---|
| Minimum Balance | N/A (abstract) | ₹500 | ₹1000 |
| Account Type | N/A (abstract) | "Savings" | "Current" |
| Special Feature | None | Interest calculation | Overdraft facility |
| Constructor | Validates common rules | Calls super() with type | Calls super() with type |

# Task 1: Convert Account to Abstract Class

Modify the Account class to be abstract:

java

```java
package com.gdb.domain;

import com.gdb.exceptions.*;

public abstract class Account {

    // ===== Constants =====
    private static final int MIN_AGE = 18;
    private static final int MIN_PIN = 1000;
    private static final int MAX_PIN = 9999;

    // ===== Fields =====
    private int accountNumber;
    private String name;
    private int age;
    private double balance;
    private String status;
    private Integer pin;

    // ===== Abstract Methods =====
    public abstract double getMinimumBalance();
    public abstract String getAccountType();

    // ===== Constructor =====
    public Account(int accountNumber, String name, int age,
                   double initialBalance)
                   throws IllegalArgumentException {

        // Validate age
        if (age < MIN_AGE) {
            throw new IllegalArgumentException(
                "Customer must be at least " + MIN_AGE + " years old. Provided: " + age
            );
        }
```

```
        // Validate minimum balance (delegated to subclass)
        double minBalance = getMinimumBalance();
        if (initialBalance < minBalance) {
            throw new IllegalArgumentException(
                getAccountType() + " account requires minimum balance of ₹" + minBalance +
                ". Provided: ₹" + initialBalance
            );
        }

        // Initialize fields
        this.accountNumber = accountNumber;
        this.name = name;
        this.age = age;
        this.balance = initialBalance;
        this.status = "Active";
        this.pin = null;
    }

    // ===== Other Methods (Same as before) =====
    // ... deposit, withdraw, closeAccount, reopenAccount, etc.
    // ... all getters and setters
}
```

# Task 2: Create SavingsAccount Class

```
package com.gdb.domain;
import com.gdb.exceptions.*;

public SavingsAccount extends Account {

    // ===== Constants =====
    private static final double MINIMUM_BALANCE = 500.0;
    private static final String ACCOUNT_TYPE = "Savings";
    private static final double INTEREST_RATE = 4.0; // 4% per annum
        // ===== Constructor =====
    public SavingsAccount(int accountNumber, String name, int age,
                          double initialBalance)
                          throws IllegalArgumentException {
        super(accountNumber, name, age, initialBalance);
    }
        // ===== Abstract Method Implementations =====
    @Override
    public double getMinimumBalance() {
        return MINIMUM_BALANCE;
    }
    @Override
    public String getAccountType() {
        return ACCOUNT_TYPE;
    }
    // ===== Savings-Specific Methods =====
    public double calculateInterest(int years) {
        if (years < 0) {
            throw new IllegalArgumentException("Years must be non-negative");
        }
        return getBalance() * (INTEREST_RATE / 100) * years;
    }
        public double getInterestRate() {
        return INTEREST_RATE;
    }
}
```

# Task 3: Create CurrentAccount Class

java

```java
package com.gdb.domain;

import com.gdb.exceptions.*;

public CurrentAccount extends Account {

    // ===== Constants =====
    private static final double MINIMUM_BALANCE = 1000.0;
    private static final String ACCOUNT_TYPE = "Current";
    private static final double OVERDRAFT_LIMIT = 5000.0;

    // ===== Fields =====
    private double overdraftUsed;

    // ===== Constructor =====
    public CurrentAccount(int accountNumber, String name, int age,
                          double initialBalance)
                          throws IllegalArgumentException {
        super(accountNumber, name, age, initialBalance);
        this.overdraftUsed = 0.0;
    }

    // ===== Abstract Method Implementations =====
    @Override
    public double getMinimumBalance() {
        return MINIMUM_BALANCE;
    }

    @Override
    public String getAccountType() {
        return ACCOUNT_TYPE;
    }

    // ===== Override Withdraw for Overdraft =====
    @Override
    public void withdraw(double amount, int pin)
```

```
        throws InvalidAmountException,
               InsufficientBalanceException,
               MinimumBalanceViolationException,
               InactiveAccountException,
               InvalidPinException {

    // Call parent validation (active, PIN, amount)
    validateActive();
    validatePin(pin);
    validateAmount(amount);

    // Check with overdraft
    double availableBalance = getBalance() - getMinimumBalance() + OVERDRAFT_LIMIT - overdraftU
sed;
    if (amount > availableBalance) {
        throw new InsufficientBalanceException(
            "Insufficient funds. Available: ₹" + availableBalance +
            " (including ₹" + OVERDRAFT_LIMIT + " overdraft), Requested: ₹" + amount
        );
    }

    // Apply overdraft if balance goes below minimum
    double newBalance = getBalance() - amount;
    if (newBalance < getMinimumBalance()) {
        double overdraftAmount = getMinimumBalance() - newBalance;
        this.overdraftUsed += overdraftAmount;
    }

    // Update balance using parent's field (access via setter or protected)
    // Note: You'd need to make balance protected or add setBalance()
    // For now, we'll use a protected setter method
    setBalance(newBalance);
    updateDailyWithdrawalTotal(amount);
}

// ===== Current-Specific Methods =====
public double getOverdraftLimit() {
    return OVERDRAFT_LIMIT;
}
```

```
    public double getOverdraftUsed() {
        return overdraftUsed;
    }

    public double getAvailableOverdraft() {
        return OVERDRAFT_LIMIT - overdraftUsed;
    }

    public boolean isUsingOverdraft() {
        return overdraftUsed > 0;
    }

    public void repayOverdraft(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Repayment amount must be positive");
        }
        if (amount > overdraftUsed) {
            throw new IllegalArgumentException(
                "Amount exceeds overdraft used (₹" + overdraftUsed + ")"
            );
        }
        this.overdraftUsed -= amount;
        setBalance(getBalance() + amount);
    }
}
```